Documentação - Fase II TP

Alunos:

- Felipe Portes Antunes
- Lucas Teixeira Reis
- Thayná Andrade Caldeira Antunes

Professor: Walisson Ferreira de Carvalho

Link do Projeto: Checklist App

Link do vídeo desta etapa: Exemplo de funcionamento do Projeto

Link para a fase I do projeto: Fase I do Projeto

A. Qual a estrutura usada para representar os registros?

1. Interface Registro (Contrato)

Define as operações básicas que todo registro deve implementar para garantir a persistência.

- **setId(int) e getId():** Gerenciamento de identificadores únicos.
- **toByteArray():** Serialização do objeto para o formato de bytes.
- **fromByteArray(byte[]):** Desserialização para reconstruir o objeto a partir dos bytes.

2. Classe RegistroIndex(Índice)

Registro auxiliar utilziado para otimizar as buscas no sistema:

- **Mapeamento:** Armazena o par (id, endereco).
- **Integração:** Utilizado pelo Hash Extensível para localizar rapidamente a posição de um registro no arquivo principal.

3. Classe Arquivo<T>(Peristência Genérica)

Gerencia o armazenamento físico em disco com suporte a tipos genéricos. O arquivo é estruturado da seguinte forma:

Cabeçalho (12 bytes fixos)

- **Último ID usado (4 bytes):** Mantém o controle do incremento de IDs.
- **Ponteiro para primeiro deletado (8 bytes):** Endereço do início da lista de registros excluídos.

Estrutura de cada Registro

- **Lápide (1 byte):** Indica se o registro está ativo (' ') ou foi deletado ('*').
- **Tamanho (2 bytes):** Indica o comprimento dos dados do registro.
- **Dados serializados (Variável):** Conteúdo binário da entidade.

4. Gerenciamento de Espaço e Exclusão

- **Exclusão Lógica:** O sistema marca a lápide sem remover o dado fisicamente de imediato.
- **Reutilização de Espaço:** Mantém uma lista ligada de espaços deletados para que novas inserções ocupem esses "buracos", otimizando o tamanho do arquivo.
- **Flexibilidade:** Funciona com qualquer classe que implemente a interface Registro (como Usuario, Tarefa, Categoria, Tag e LogConclusao).

B. Como atributos multivalorados do tipo string foram tratados?

1. Abordagem Geral

No projeto, atributos multivalorados não são armazenados como coleções diretas (como `List<String>`) dentro da entidade principal. Em vez disso, foram adotadas duas estratégias distintas dependendo da necessidade de busca e organização.

2. Relacionamento entre Entidades (N:N)

Para casos onde as strings representam entidades independentes que precisam de organização (como as Tags de uma Tarefa), o sistema utiliza uma **Tabela de Associação**.

- **Entidade de Associação:** Foi criada a classe `TarefaTag` .
- **Campos:** Armazena os pares `id_tag` e `id_tarefa` .
- **Funcionamento:** Isso transforma o atributo multivalorado em um relacionamento N:N (muitos-para-muitos). As tags são registros individuais em seu próprio arquivo, relacionados à tarefa através dessa ponte.

3. Atributo Simples Concatenado

Para casos onde os dados são informativos e não precisam de indexação individual complexa, utilizou-se a simplificação por texto.

- **Campo:** `LogConclusao.resumo_tags` .
- **Estrutura:** É uma string única que contém o texto concatenado das tags.
- **Persistência:** Gravado e recuperado utilizando os métodos `DataOutputStream.writeUTF` e `DataInputStream.readUTF` .

4. Conclusão da Estratégia

- **Multivalorados estruturados:** Tratados via relacionamento entre tabelas (`TarefaTag`).
- **Multivalorados descritivos:** Tratados como uma string única de texto simples (`resumo_tags`), servindo apenas como uma descrição ou log, sem ser uma coleção serializada de strings.

C. Como foi implementada a exclusão lógica?

1. Mecanismo de Lápide

A exclusão lógica é gerenciada na classe `Arquivo<T>` através de um byte de controle chamado "lápide", localizado no início de cada registro:

- **Byte ' ' (espaço):** Indica um registro ativo.
- **Byte '*' (asterisco):** Indica um registro deletado.

2. Fluxo de Exclusão

O método `Arquivo.delete(int id)` opera da seguinte forma:

1. **Navegação:** Percorre o arquivo sequencialmente a partir do byte 12 (ignorando o cabeçalho).
2. **Leitura:** Para cada registro, o sistema lê a lápide, o tamanho e os dados serializados.
3. **Identificação:** Se encontrar um registro ativo (' ') com o ID correspondente:
   - O ponteiro de escrita volta para a posição inicial do registro.
   - O caractere '*' é gravado na lápide.
   - O método `addDeleted(tamanho, posicao)` é acionado para gerenciar o espaço liberado.

3. Reaproveitamento de Espaço (Lista Encadeada)

Em vez de simplesmente descartar o espaço, o sistema utiliza uma lista ligada de registros deletados para otimizar o uso do disco:

- **Cabeçalho:** O arquivo mantém um ponteiro (8 bytes) no cabeçalho que aponta para o primeiro registro deletado.
- **Estrutura do Bloco Livre:** No espaço do registro deletado, logo após o tamanho, o sistema grava o ponteiro para o próximo registro da lista de excluídos.
- **Inserção:** Novos registros consultam essa lista para verificar se podem ocupar um "buraco" existente antes de expandir o final do arquivo.

4. Aplicação no Método Update

O mecanismo de exclusão lógica também é fundamental para o `Arquivo.update(...)`:

- **Crescimento de Dados:** Se um registro for atualizado e seu novo tamanho for maior que o espaço original, o registro antigo é marcado como deletado ('*').
- **Relocação:** O registro antigo entra para a lista de espaços livres, e a nova versão do registro é gravada em um espaço reutilizável compatível ou no final do arquivo.

D. Além das PKs, quais outras chaves foram utilizadas nesta etapa?

1. Chaves Estrangeiras (FKs)

Foram utilizadas para modelar as dependências e integridade referencial entre as entidades principais:

- `Categoria.id_user`: Referencia a entidade Usuario.
- `LogConclusao.id_tarefa`: Referencia a entidade Tarefa.
- `TarefaTag.id_tag` e `TarefaTag.id_tarefa`: Relacionam as entidades Tag e Tarefa.

2. Chaves de Arquivos de Relacionamento (Índices)

Utilizadas pelos gerenciadores (Managers) para mapear coleções de um para muitos (1:N), permitindo que o sistema localize listas de registros vinculados a um ID pai:

- **UsuarioCategoriasManager:** Usa o `idUsuario` como chave para recuperar a lista de categorias do usuário.
- **UsuarioTarefasManager:** Usa o `idUsuario` como chave para recuperar a lista de tarefas do usuário.
- **CategoriaTarefasManager:** Usa o `idCategoria` como chave para recuperar a lista de tarefas de uma categoria específica.

- **TarefaLogsManager:** Usa o `idTarefa` como chave para recuperar o histórico de logs de uma tarefa.

3. Chave Composta (TarefaTag)

Para o relacionamento N:N entre Tarefa e Tag, foi implementada uma chave composta para garantir a unicidade do par:

- **Cálculo:** O identificador é gerado pela fórmula `idTarefa * 1000000 + idTag`.
- **Objetivo:** Garante que uma mesma tag não seja vinculada à mesma tarefa mais de uma vez na persistência.

4. Resumo das Chaves

- **Chaves Estrangeiras:** Modelagem de dependência direta entre entidades.
- **Chaves de Relação:** Mapeamento de coleções e listas vinculadas.
- **Chave Composta:** Garantia de unicidade em relacionamentos muitos-para-muitos.

E. Como a estruturas (hash) foi implementada para cada chave de pesquisa?

1. Estrutura Genérica: HashExtensivel

O projeto utiliza uma única implementação de **Hash Extensível** (dynamic hashing) que é persistente em arquivo e se adapta conforme o volume de dados cresce.

Formato do Arquivo de Índice

O arquivo é estruturado em três partes principais:

- **Cabeçalho:** Armazena a profundidade global, o tamanho de cada bucket e o tamanho total do diretório.
- **Diretório:** Uma lista de ponteiros que indicam em qual bucket cada entrada deve ser armazenada.
- **Buckets:** Unidades de armazenamento que guardam a profundidade local, a quantidade atual de elementos e os pares **(chave, endereço)**.

Funcionamento do Algoritmo

- **Função Hash:** Utiliza a própria chave inteira.
- **Seleção de Bucket:** O índice é calculado através da operação: `indice = hash % (1 << profundidadeGlobal)`.
- **Entrada:** Cada item no bucket é composto por um `int chave` e um `long endereco`.

2. Tratamento de Overflow (Bucket Cheio)

Quando um bucket atinge sua capacidade máxima, a classe `HashExtensivel` realiza o seguinte fluxo:

1. **Divisão:** Aumenta a profundidade local do bucket cheio.
2. **Expansão:** Se a profundidade local ultrapassar a global, o diretório dobra de tamanho.
3. **Redistribuição:** Dois novos buckets são criados e as entradas são reorganizadas com base no novo bit de índice.

3. Implementações Específicas

A estrutura genérica foi especializada para atender às diferentes necessidades de busca do sistema:

HashIndexUsuario

- **Chave de pesquisa:** `idUsuario`.
- **Valor armazenado:** `RegistroListaEnderecos` (ID do usuário + lista de endereços físicos das tarefas).
- **Objetivo:** Buscar instantaneamente todas as tarefas vinculadas a um usuário específico.
- **Arquivo:** `./dados/idx_usuario_tarefa_hash.db`.

HashIndexTarefaTags

- **Chave de pesquisa:** `idTarefa`.
- **Valor armazenado:** `RegistroListaTags` (ID da tarefa + lista de IDs das tags associadas).
- **Objetivo:** Recuperar rapidamente todas as tags que pertencem a uma tarefa.
- **Arquivo:** `./dados/idx_tarefa_tags_hash.db`.

4. Operações Suportadas

Cada índice implementado oferece as seguintes funcionalidades básicas:

- **Inserir:** Adiciona um novo par chave-valor e gerencia possíveis divisões de bucket.
- **Buscar:** Localiza o endereço ou valor associado a uma chave específica.
- **Remover:** Exclui a chave do índice.
- **BuscarTodos:** Retorna a lista completa de valores associados àquela chave de pesquisa.

Resumo: O sistema centraliza a complexidade da indexação em uma classe robusta de Hash Extensível, especializando-a para garantir que buscas por Usuários e Tarefas sejam realizadas em tempo constante, independentemente do crescimento do arquivo de dados.

F. Como foi implementado o relacionamento 1:N (explique a lógica da navegação entre registros e integridade referencial)?

1. Estrutura de Gerenciamento

Diferente de bancos de dados relacionais convencionais, este projeto não utiliza apenas uma coluna de Chave Estrangeira (FK). O relacionamento 1:N é implementado através de **Gerenciadores Específicos (Managers)**, que funcionam como índices de associação:

- **UsuarioTarefasManager:** Relaciona um Usuário às suas diversas Tarefas.
- **UsuarioCategoriasManager:** Relaciona um Usuário às suas Categorias.
- **CategoriaTarefasManager:** Relaciona uma Categoria às suas Tarefas.

Cada gerenciador utiliza um arquivo de índice onde a **Chave** é o ID do pai e o **Valor** é um registro contendo uma lista de IDs dos filhos (`List<Integer>`).

2. Lógica de Navegação e Busca

A recuperação dos dados ocorre em duas etapas para garantir a modularidade:

1. **Obtenção de IDs:** O DAO chama o Manager correspondente (ex: `usuarioTarefasManager.buscarTarefasDoUsuario(idUser)`) para obter a lista de IDs vinculados.
2. **Carregamento de Objetos:** Para cada ID retornado, o sistema realiza uma busca individual no arquivo de dados da entidade (ex: `buscarTarefa(id)`) para reconstruir o objeto completo.

3. Armazenamento e Atualização

O armazenamento da lista de filhos é dinâmico e persistente:

- **Adição:** Ao vincular um novo filho (ex: nova Tarefa), o sistema lê o registro da lista do pai, adiciona o novo ID e executa um `create` ou `update` no índice.
- **Remoção/Troca:** Se uma tarefa muda de categoria, o sistema remove o ID da lista da categoria antiga e o adiciona na lista da nova categoria.

4. Integridade Referencial (Controle Manual)

Como não há um motor de banco de dados automático, a integridade é garantida via código dentro das classes DAO:

Operação	Regra de Integridade
Inclusão	Verifica obrigatoriamente se o Pai (Usuário/Categoria) existe antes de salvar o filho.
Exclusão de Usuário	Bloqueada se o usuário possuir tarefas ou categorias vinculadas.
Exclusão de Categoria	Bloqueada se houver qualquer tarefa associada a ela.
Exclusão de Tarefa	Bloqueada se houver logs ou tags vinculados; remove as referências nos Managers antes de apagar.

5. Resumo da Implementação

- **Persistência:** Baseada em mapeamento direto `Pai -> [Lista de Filhos]`.
- **Desempenho:** O uso de índices específicos evita a varredura completa do arquivo de dados para encontrar registros relacionados.
- **Consistência:** Mantida rigorosamente pela lógica de negócio nos DAOs, impedindo estados órfãos no sistema.

G. Como os índices são persistidos em disco? (formato, atualização, sincronização com os dados).

1. Formato em Disco (Estrutura Binária)

Os índices utilizan a estrutura de **Hash Extensível** persistente. O arquivo é organizado em três camadas lógicas para permitir o acesso rápido:

Cabeçalho (12 bytes fixos)

- **Profundidade Global (4 bytes):** Controla o tamanho atual do diretório.
- **Tamanho do Bucket (4 bytes):** Define quantos registros cabem em cada balde.
- **Tamanho do Diretório (4 bytes):** Indica a quantidade de ponteiros de buckets.

Diretório (Variável)

- Composto por uma lista de **ponteiros (8 bytes cada)** que direcionam a busca para o endereço físico do bucket correspondente no arquivo.

Buckets (Variável)

Cada bucket contém os metadados de controle e os dados indexados:

- **Profundidade Local (4 bytes):** Nível de refino do bucket atual.
- **Quantidade (4 bytes):** Número de registros presentes no bucket.
- **Pares Chave-Endereço (12 bytes cada):**
  - **Chave (4 bytes):** Identificador único (ID).
  - **Endereço (8 bytes):** Offset (posição) do registro no arquivo de dados principal.

2. Fluxo de Atualização

A atualização ocorre de forma coordenada entre o arquivo de dados e o arquivo de índice:

- **Inserção:**
  i. O registro é gravado no final do arquivo de dados.
  ii. O endereço gerado (offset) é enviado para o `HashExtensivel.inserir()`.
  iii. O índice calcula o bucket destino e armazena o par `(chave, endereco)`. Se o bucket estiver cheio, ocorre a divisão e redistribuição automática.
- **Remoção:**
  i. O sistema localiza o bucket pela chave.
  ii. O par `(chave, endereco)` é removido do índice.
  iii. No arquivo de dados, o registro é marcado apenas com a lápide de exclusão lógica.
- **Atualização:**
  i. Se o novo dado couber no espaço original, o endereço no índice permanece o mesmo.
  ii. Se o dado crescer e precisar de um novo local, o índice é atualizado com o **novo endereço** do registro.

3. Sincronização com os Dados

A integridade entre o índice e o dado real é mantida através de mecanismos de controle na classe `ArquivoIndex<T>`:

1. **Endereços como Ponteiros:** O índice nunca armazena o objeto, apenas o "ponteiro" (offset) para onde ele está no disco.
2. **Sincronização Automática:** Cada operação de CRUD (Create, Read, Update, Delete) no `ArquivoIndex` garante que a alteração no dado seja refletida no índice no mesmo ciclo de execução.
3. **Reaproveitamento de Espaço:** Quando um registro é excluído logicamente ou movido por falta de espaço, o sistema utiliza uma lista ligada de espaços vazios para futuras gravações, mantendo o arquivo de dados compacto sem corromper os índices existentes.

4. Resumo Técnico

- **Busca:** Tempo médio **O(1)** para acesso direto por chave.
- **Escalabilidade:** O diretório cresce dinamicamente conforme a necessidade, dobrando de tamanho apenas quando a profundidade local de um bucket cheio iguala a profundidade global.
- **Confiabilidade:** O uso de `ArquivoIndex<T>` encapsula a lógica de dados e índices, tratando-os como uma unidade atômica de sincronização.

H. Como está estruturado o projeto no GitHub (pastas, módulos, arquitetura)?

1. Arquitetura Geral

O projeto é um aplicativo Java puro para gerenciamento de tarefas (checklist). Sua principal característica é a **persistência customizada**: não utiliza bancos de dados relacionais (SQL) ou externos. Toda a lógica de armazenamento é feita através de arquivos binários e índices de Hash Extensível implementados do zero.

2. Estrutura de Pastas e Arquivos

```
checklist-app/
├── docs/                # Documentação técnica e manuais
│   ├── fase-i.md        # Relatório da primeira etapa
│   ├── aux/             # Materiais auxiliares sobre arquivos
│   ├── images/          # Assets da documentação
│   └── pdf/             # Documentos exportados
├── oracleJdk-26/        # Ambiente Java (JDK 26) incluso
├── target/              # Artefatos gerados pelo Maven (build)
├── pom.xml              # Dependências e configurações do projeto
├── README               # Guia rápido de uso
└── [Raiz dos Fontes]    # Código-fonte principal (.java)
```

3. Organização dos Módulos (Camadas)

O código está organizado seguindo uma arquitetura em camadas para separar responsabilidades:

Camada de Domínio (Entidades)

Classes que representam os dados do sistema.

- `Usuario` , `Tarefa` , `Categoria` , `Tag` , `LogConclusao` .

Camada de Persistência (Core)

O "motor" de armazenamento do projeto.

- `Registro` : Interface base para todos os objetos persistíveis.
- `Arquivo<T>` : Gerenciador genérico de registros em disco.
- `ArquivoIndex` : Especialização para lidar com índices.
- `HashExtensivel` : Implementação da estrutura de dados de busca.

Camada de Acesso a Dados (DAOs)

Interface entre o domínio e os arquivos físicos.

- `UsuarioDAO` , `TarefaDAO` , `CategoriaDAO` , `TagDAO` , `LogConclusaoDAO` .

Camada de Relacionamentos

Gerenciadores específicos para manter a integridade 1:N e N:N.

- `TarefaTagDAO`, `UsuarioTarefasManager`, `UsuarioCategoriasManager`, `CategoriaTarefasManager`, `TarefaLogsManager`.

Camada de Apresentação (Interface)

Interação com o usuário via console ou interface gráfica.

- `App.java` / `Principal.java`: Pontos de entrada.
- `MenuUsuarios`, `MenuTarefas`, `MenuCategorias`, `MenuTags`, `MenuLogs`.

4. Diferenciais do Projeto

- **Independência de Banco de Dados:** Todo o gerenciamento de bytes, ponteiros e endereços de memória em disco é feito manualmente.
- **Eficiência de Busca:** Utilização de Hash Extensível para garantir busca em tempo constante.
- **Integridade Manual:** Como não há chaves estrangeiras automáticas, os DAOs verificam as relações antes de qualquer inserção ou exclusão.
- **Maven:** Utilizado para padronizar o build e a compilação, facilitando a portabilidade do projeto.